

RAPPORT TECHNIQUE

Architecture, Fonctionnalités & Sécurité
de l'application Qalami

Espaces Superadmin, Admin, Enseignant & Sécurisation des Actions Serveur

Auteur : Équipe de Développement & Sécurité ADST

Version : 1.0

Date de génération : 01/06/2026

Statut : Document de Référence Sécurisé

1. Vue d'ensemble de l'application Qalami

L'application Qalami est une plateforme intégrée de gestion scolaire multi-établissement. Elle a été conçue pour centraliser et simplifier la communication, le suivi académique, la gestion administrative et le contrôle financier au sein des écoles primaires et secondaires.

La plateforme est structurée autour de trois espaces principaux s'adaptant aux différents acteurs : l'espace Superadmin (gestionnaire général de la plateforme), l'espace Admin (le directeur d'école et son secrétariat) et l'espace Enseignant (le personnel pédagogique). Un cloisonnement rigide par établissement garantit qu'aucune information ne filtre entre les écoles indépendantes.

2. Espace Superadmin (Gestion Globale)

L'espace Superadmin est le poste de contrôle de l'ensemble de la solution Qalami. Ses fonctions couvrent :

- **Gestion des Établissements (Schools)** : Enregistrement des nouvelles écoles partenaires, activation ou désactivation des licences d'exploitation, et suivi du forfait associé.
- **Gestion de la Sécurité des Comptes** : Capacité de réinitialiser le mot de passe d'un utilisateur de n'importe quel rôle (directeur, professeur) ou de suspendre un compte en cas d'activité suspecte.
- **Dashboard de Supervision** : Métriques agrégées sur l'ensemble de l'infrastructure, notamment le nombre d'écoles, le total des enseignants et élèves inscrits, et les flux de trésorerie issus des abonnements.

3. Espace Admin / Directeur d'École

L'espace Admin permet à chaque directeur d'école de piloter sa structure de manière autonome et sécurisée. Cet espace intègre les modules essentiels suivants :

- **Gestion Académique** : Configuration des classes, des niveaux (ex. collège, primaire), des matières enseignées et de leurs coefficients respectifs. L'administrateur peut adapter les coefficients par classe pour refléter les programmes spécifiques.
- **Administration des Élèves, Professeurs et Parents** : Inscription des élèves, liaison avec les comptes parents correspondants, et affectation des professeurs aux classes et matières.
- **Suivi Financier & Comptabilité** : Planification des frais scolaires mensuels, suivi en temps réel des règlements (payé, en retard), et envoi instantané de rappels de paiement et de notifications d'impayés.
- **Journal d'Activité (Activity Log)** : Audit trail recensant de manière transparente les actions majeures effectuées par le personnel administratif, indiquant précisément l'identité de l'auteur (nom et numéro de téléphone) pour prévenir les abus.

4. Espace Enseignant (Teacher)

L'espace Enseignant est axé sur la gestion de classe au jour le jour, fournissant des outils pragmatiques et mobiles :

- **Emploi du Temps Quotidien** : Visualisation rapide des cours programmés avec affichage de la classe, de la matière et de la salle de classe.
- **Suivi Éducatif & Observations** : Saisie d'observations comportementales et de remarques scolaires visibles par l'administration et les parents d'élèves.
- **Gestion des Notes** : Enregistrement direct des notes des devoirs et examens des classes assignées.
- **Mise à jour du Profil** : Modification sécurisée de ses données personnelles (numéro de téléphone, avatar) et mise à jour de son mot de passe de connexion.

5. Architecture Technique

L'application Qalami est construite sur une architecture découplée de type Jamstack hybride :

- **Framework Frontend** : Next.js 16.1.4 utilisant l'App Router. Cette structure facilite la navigation fluide, les layouts imbriqués, et le rendu hybride (Server-Side Rendering pour les données dynamiques et Static Site Generation pour les éléments statiques).
- **Backend et Services** : Supabase, qui fournit une base PostgreSQL relationnelle robuste, le moteur d'authentification des utilisateurs (Supabase Auth) ainsi que le stockage de fichiers statiques (Supabase Storage).
- **Styles et Interface** : Tailwind CSS v4 combiné avec Radix UI pour la création de composants complexes (boîtes de dialogue, formulaires). Framer Motion ajoute des micro-animations interactives de haute qualité.

6. Liste des Outils & Packages Utilisés

Voici un aperçu des dépendances majeures déclarées dans le fichier package.json :

Nom du Package	Version	Rôle principal dans l'application
next	16.1.4	Framework d'application React et serveur de rendu (SSR).
@supabase/supabase-js	2.91.1	SDK d'intégration avec les bases PostgreSQL et l'Auth.
jspdf	4.2.1	Moteur de génération de documents PDF (dont ce rapport).
framer-motion	12.29.2	Moteur d'animations interactives et de transitions.
recharts	3.7.0	Graphiques interactifs des indicateurs et statistiques.
react-hook-form	7.71.1	Formulaires de saisie performants et typés.
zod	3.25.76	Déclaration de schémas de validation et typage statique.

7. Politique de Sécurité des Données & des Rôles

La protection des données dans Qalami est structurée en trois couches étanches :

7.1 Sécurisation des Accès aux Pages (Middleware)

Le middleware Next.js intercepte chaque requête de page. Il décode le jeton JWT fourni par Supabase Auth pour s'assurer que la session est valide et redirige l'utilisateur vers la page de login en cas d'absence de session. De plus, les layouts d'espaces (ex. /admin, /teacher, /super-admin) effectuent un contrôle d'accès basé sur les rôles (RBAC) pour rejeter immédiatement les utilisateurs n'ayant pas les droits requis.

7.2 Sécurité au Niveau des Lignes PostgreSQL (RLS)

Le Row Level Security (RLS) est actif sur l'ensemble de la base de données. Chaque table contient une colonne `school_id`. Les requêtes exécutées par l'intermédiaire du client Supabase standard filtrent automatiquement les lignes pour ne retourner que celles correspondant à l'établissement de l'utilisateur connecté (`school_id`), interdisant toute fuite inter-écoles.

7.3 Sécurisation des Actions Serveur (Server Actions)

Les Server Actions de Next.js sont par nature des endpoints publics exécutés à distance. Pour réaliser des opérations complexes sans être bloqué par les restrictions de lecture/écriture du client standard, nous utilisons un client d'administration Supabase (`createAdminClient`) utilisant la clé secrète `service_role`, qui contourne le RLS.

Sans précaution, ce contournement peut être détourné : un utilisateur authentifié pourrait appeler ces fonctions en transmettant des identifiants (`teacherId`, `schoolId`, ou `studentId`) extérieurs à son école. C'est pourquoi nous avons mis en place une validation manuelle systématique.

8. Corrections de Sécurité Appliquées

Dans le cadre de cette mise à jour, nous avons sécurisé plusieurs Server Actions critiques qui utilisaient `createAdminClient()` afin d'empêcher les failles d'escalade de privilèges et de fuites de données :

Vulnérabilité Corrigée : Contournement de RLS via Server Actions

Les Server Actions exposées par Next.js étaient vulnérables à la manipulation d'arguments. En transmettant des identifiants de professeurs, d'élèves ou d'écoles tiers, des utilisateurs connectés pouvaient accéder ou modifier des données confidentielles d'autres établissements. La correction consiste à authentifier l'utilisateur via le serveur et valider ses autorisations par rapport à son établissement avant d'exécuter la requête d'administration.

8.1 Fichier `src/app/teacher/actions.ts` (Espace Enseignant)

- **`getTeacherScheduleAction`** : Valide que l'appelant est authentifié. Si son rôle est 'teacher', il ne peut demander que son propre emploi du temps (`teacherId === user.id`). Pour les autres rôles (admin, school_staff), le système valide l'habilitation.
- **`getTeacherAssignmentsAction`** : Même validation, empêchant un enseignant de consulter la liste des classes et matières assignées à un autre collègue.
- **`getClassStudentsAction`** : Avant de retourner les données confidentielles des élèves d'une classe, le système vérifie si l'enseignant appelant est affecté à cette classe dans 'teacher_assignments'. Si ce n'est pas le cas, l'action est immédiatement rejetée.

- **createRemarkAction** : Empêche un enseignant d'ajouter des remarques sur un élève sous l'identité d'un autre enseignant (validation que le `teacher_id` du payload correspond à l'ID de l'utilisateur connecté).

8.2 Fichier `src/app/admin/actions.ts` (Espace Administrateur)

- **getSchoolLinkedProfileIds** : Utilise `getMySchoolContext()` pour vérifier que l'administrateur connecté appartient bien à l'école dont il demande les identifiants de profils.
- **secureFetchProfiles** : Restreint les profils récupérés à ceux appartenant à la même école que l'administrateur connecté (par croisement avec `profiles` ou `profile_schools`), éliminant silencieusement les identifiants tiers.
- **getSchoolMetricsCounts** : Interdit la lecture des statistiques scolaires (nombre d'élèves, de parents, de professeurs) pour des établissements tiers.
- **notifyLateParentAction** : Récupère la fiche de l'élève ciblé par l'impayé pour valider que sa colonne `school_id` correspond bien à celle de l'administrateur connecté, évitant l'envoi de rappels à des parents d'autres écoles.
- **getAdminNotifications & getAdminUnreadNotificationsCount** : Filtrent strictement la lecture des notifications d'établissement par rapport au `school_id` de l'administrateur en session.

Résultat des Corrections

Toutes les routes d'API serveur (Server Actions) associées aux enseignants et aux administrateurs disposent désormais d'une double barrière : validation de l'identité côté serveur et cloisonnement strict au niveau de l'école ou de l'affectation pédagogique. La sécurité globale de l'application face à d'éventuelles attaques par IDOR (Insecure Direct Object Reference) est entièrement verrouillée.

9. Recommandations de Développement Futures

Afin de maintenir ce standard de sécurité de haut niveau dans les développements futurs de l'application Qalami, il est conseillé de suivre les directives suivantes :

- **Privilégier le RLS** : N'utiliser `createAdminClient()` que lorsque c'est strictement indispensable (ex. actions d'administration de la console Superadmin ou opérations impliquant Supabase Auth). Pour toutes les requêtes courantes, le client Supabase standard (`createClient()`) doit être employé afin de bénéficier des filtres RLS automatiques.
- **Validation Systématique** : Ne jamais faire confiance aux paramètres d'identité transmis par le client. Toujours récupérer l'identité en session avec `supabase.auth.getUser()` côté serveur.
- **Journalisation des Anomalies** : Enregistrer dans le journal d'activité les tentatives d'appel avec des paramètres incohérents (ex. un utilisateur d'une école demandant une ressource d'une autre école) pour identifier d'éventuels comportements malveillants.